研究計画書（改訂版）

Federated Learning における悪性更新の検知可能性に関する研究

— データ分布の不均一性と攻撃のステルス性を軸とした更新ベクトルレベルの評価 —

3CDA2216 笠原秀星

2026 年 9 月 1 日

1 本計画の位置づけ

1.1 これまでの経緯

これまで著者は，Fang ら [1] の Adaptive Model Poisoning Attack を再実装し，FedAvg，Median，Trimmed Mean，Krum，Bulyan の 5 種類の集約手法に対する耐性を，IID/Non-IID 環境・悪性クライアント割合を振ったスイープ（38 条件 ×20 trial，計 760 回）によって評価してきた．しかし，計算資源の制約から通信ラウンド数を 20 に制限したため，攻撃なしのベースラインですら学習が収束せず（IID で Accuracy 0.4 前後，Krum・Non-IID で 0.14），最終精度の絶対値・手法間の順位のいずれも結論できないという根本的な問題が残った．収束させるには Fang らと同等の 2,000 イテレーション相当が必要であり，現有の CPU 環境と残り時間では実行不可能である．

1.2 方針転換の理由

上記の行き詰まりは研究テーマ自体の問題ではなく，**「最終精度」を主評価指標に据えたことに起因する**．最終精度は学習の収束を前提とするため計算コストが高い．一方，攻撃と防御の本質的な攻防は，各ラウンドでサーバが集約を行う直前の**更新ベクトル空間**で生じている．そこで本改訂計画では，評価の視点を最終精度から更新ベクトルレベルへ移し，防御を「悪性更新を正常更新から見分けられるか」という**二値分類（異常検知）問題**として定式化して評価する．検知性能はモデルが最終精度に到達していなくても各ラウンドで測定できるため，収束を必要とせず，小規模・短ラウンドの実験でも科学的に妥当な結論が得られる．

本計画は，Fang らの特定の実験設定（100 クライアント・2,000 イテレーション・特定の攻撃・38 条件スイープ）から独立し，FL における攻撃・防御一般を対象とする．これまでに構築した Flower + PyTorch の実験基盤，集約手法の実装（`aggregation.py`），攻撃の実装（`attack.py`），スイープ・作図スクリプトはそのまま再利用する．

2 研究の問いと仮説

2.1 研究の問い

1. **RQ1．** FL における悪性更新は，更新ベクトルレベルの異常検知でどの程度見分けられるか．検知性能は集約手法非依存の代表的な検知器の間でどう異なるか．
2. **RQ2．** データ分布の不均一性（Non-IID 度）は検知性能をどう低下させるか．その低下は学習ダイナミクスとは独立に生じるか．
3. **RQ3．** 攻撃のステルス性（悪性更新を正常クラスタ内に収める度合い）は検知性能をどう左右するか．検知が破綻する境界はどこか．

2.2 中心仮説

> 悪性更新が検知可能かどうかは，**悪性更新の押し出し量が正常クライアント間のばらつきを超えているか**でほぼ決まる．Non-IID は正常更新のばらつきを増大させるため，学習ダイナミクスとは独立に，攻撃を正常な多様性の中へ紛れ込ませ，検知性能を低下させる．

この仮説を検証するための説明変数として，正常クラスタの広がり spread，悪性更新の押し出し距離 push，およ

びその比 push_ratio （§4.5 で定義）を導入する．仮説が正しければ，検知性能（ROC-AUC）は push_ratio が 1 を下回る領域で急落し，Non-IID 度が強いほど同じ攻撃強度でも push_ratio が小さくなる，という関係が観測されるはずである．

3 脅威モデル

- **攻撃者の知識：** full knowledge を仮定する．悪性クライアントは各ラウンドで正常クライアントの更新を参照でき，互いに結託できる．本実験ではこれをサーバ側で模擬する（既存実装を踏襲）．
- **悪性クライアント：** 各ラウンドで固定された集合．割合は $f/n \leq 0.3$.
- **防御者：** サーバ側に位置し，全クライアントの更新ベクトルを観測できる．**クリーンな検証データは持たない**（FLTrust[14] 型の信頼ブートストラップは本研究の対象外とする）．
- **正解ラベルの扱い：** どのクライアントが悪性かという正解は**評価専用**であり，検知器には一切与えない．
- **位置づけ：**本脅威モデルは攻撃者に有利な楽観的設定である．Shejwalkar ら [15] は現実的な制約下では poisoning の脅威が限定的であると論じているが，本研究は「検知性能の上限」を評価するものとして位置づける．

4 評価枠組み

4.1 記法

通信ラウンド t において，参加クライアント $i \in \{1, \ldots, n\}$ のモデル更新ベクトルを $g_i = w_i - w_{\text{global}}$ とする．正常クライアントの添字集合を B，悪性クライアントの添字集合を M（$|M| = f$）とする．$\bar{g}_B = \frac{1}{|B|} \sum_{i \in B} g_i$ を正常更新の重心とする．

4.2 検知器

いずれも集約手法に依存しない汎用的な検知器であり，各クライアント i に異常スコア s_i（大きいほど悪性らしい）を与える．基本の 3 種を実装し，D4 は余力に応じて追加する．

D1．ノルム偏差

$$s_i = \left| \, \|g_i\|_2 - \operatorname*{median}_j \|g_j\|_2 \, \right| \tag{1}$$

最も素朴な検知器．ノルムを大きく変える攻撃（スケーリング，ランダムノイズ）に強く，ステルス攻撃には弱いと予想される．重みクリッピング系防御（FLAME[12]）の系譜．

D2．参照方向へのコサイン距離

座標ごとの中央値 $r = \text{median}(g_1, \ldots, g_n)$ を参照方向とし，

$$s_i = 1 - \frac{g_i \cdot r}{\|g_i\|_2 \, \|r\|_2} \tag{2}$$

更新の**向き**の異常を捉える．FoolsGold[10]（クライアント履歴間のペアワイズコサイン類似度）や FLAME[12]（コサイン距離のクラスタリング）の系譜．補助的に，ペアワイズコサイン類似度行列から「他の悪性と向きが揃いすぎている」クライアントを検出する変種（結託 sybil 向け）も検討する．

D3．2 クラスタリング

特徴量 $\phi_i = \big[\, \|g_i\|_2,\ \text{(D2 のスコア)},\ \text{(第 1–2 主成分への射影)} \,\big]$ に対して k-means（$k = 2$）を適用し，小さい方のクラスタを悪性候補とみなす．スコアは正常クラスタ重心までの距離とする．Auror[18] やクラスタ化 FL[19] の系譜．

D4．スペクトル残差（余力があれば）

更新行列 $G = [g_1; \ldots; g_n]$ を中心化し，その第 1 右特異ベクトル v を求め，

$$s_i = (g_i \cdot v)^2 \tag{3}$$

とする．中央集権学習の Spectral Signatures[20] や FL の Divide-and-Conquer 防御 [7] の系譜であり，ステルス攻撃（特に min-max）がまさにこの検知を回避するよう設計されているため，攻撃との相性を見る対照として有用である．

4.3 攻撃

A1．符号反転攻撃（粗い攻撃・ベースライン）

$$g_i^{\mathrm{mal}} = -\lambda\, \bar{g}_B \qquad (i \in M,\ \lambda \geq 1) \tag{4}$$

ノルムも向きも正常から大きく外れるため，D1・D2 で容易に検知できるはずの下限ケース．

A2．ALIE（“A Little Is Enough”，ステルス攻撃・主対象）

Baruch ら [6] の攻撃．正常更新の座標ごとの平均 μ と標準偏差 σ を求め，

$$g_i^{\mathrm{mal}} = \mu - z\,\sigma \qquad (i \in M) \tag{5}$$

とする（悪性クライアントは全員同一のベクトルを送る）．z は n, f から定まる上限の範囲で探索し，実用上は $z \in [0.5, 1.5]$ を調整する．「正常のばらつきの範囲内に収まる最大の摂動」を送る攻撃であり，push_ratio ≈ 1 を人工的に作り出す．本研究の仮説検証の中心に置く．

A3．min-max ／ min-sum 攻撃（発展課題）

Shejwalkar & Houmansadr[7] の攻撃．摂動方向 ∇^p（$-\operatorname{sign}(\mu)$ または $-\sigma$）を固定し，スケール γ を二分探索で最大化する．min-max 制約は「どの悪性更新から任意の正常更新までの距離も，正常更新同士の最大距離を超えない」というもの．集約手法非依存で強力な攻撃であり，スケジュールに余裕がある場合に追加する．

なお，Fang ら [1] の攻撃や Inner Product Manipulation[8] も同じ枠組み（集約手法の特性を突く最適化型攻撃）に位置づけられるが，本研究では関連研究として言及するにとどめる．

4.4 データ分布（Non-IID）

Hsu ら [16] のディリクレ分布によるラベル分割を用いる．クラス c ごとに $p_c \sim \mathrm{Dir}(\alpha \mathbf{1}_n)$ を引き，クラス c の学習データを比率 p_c で各クライアントへ配分する．α を大きくすると IID に近づき，小さくすると各クライアントの保有クラスが偏る．本研究では $\alpha \in \{100\ (\approx \mathrm{IID}),\ 0.5,\ 0.1\}$ の 3 水準を用いる．Fang らのグループ分割と異なり，不均一性の強度を連続的に制御できる点が利点である．方法論上の参照は NIID-Bench[17].

4.5 評価指標

各ラウンド t で，全クライアントについて (s_i, y_i) の組が得られる（$y_i = 1$ なら悪性）．

- **検知性能（主指標，閾値非依存）：**ROC-AUC，Average Precision，および recall@f（スコア上位 f 件を悪性と判定したときの悪性回収率）．ラウンドごとに算出し，「AUC 対ラウンド」の推移を示すとともに，条件ごとに全ラウンドの平均 ± 標準偏差で要約する．
- **防御効用（橋渡し指標）：**スコア上位 f 件を除外して残りを平均した $\hat{g} = \mathrm{mean}(\{g_i : i \notin \text{top-}f\})$ について，真の正常重心とのずれ $\|\hat{g} - \bar{g}_B\|_2$ を測る．「検知できること」と「実際に防御できること」を接続する．
- **説明変数（仮説検証用）：**

$$\mathrm{spread} = \frac{1}{|B|} \sum_{i \in B} \|g_i - \bar{g}_B\|_2, \qquad \mathrm{push} = \frac{1}{|M|} \sum_{i \in M} \|g_i - \bar{g}_B\|_2, \tag{6}$$

$$\mathrm{push_ratio} = \frac{\mathrm{push}}{\mathrm{spread}}. \tag{7}$$

- **主図：** 横軸 push_ratio，縦軸 ROC-AUC の散布図を，点を (α，攻撃) で色分けして描く．「push_ratio が 1 を切ると AUC が崩れる」ことを 1 枚で示すことを目標とする．
- **精度確認（補助）：** 代表条件を 4–6 セルだけ選び，学習率を上げて概ね収束させ，検知の失敗が実際の精度低下・防御効用の悪化に対応することを確認する．

収束を必要としない理由と検証ゲート．検知は「そのラウンドの更新ベクトル集合が分離可能か」という問題であり，モデルが最終精度に到達している必要はない．ただし，検知性能がラウンドをまたいで極端にドリフトすると解釈が困難になるため，§7 のゲート G1 で，パイロット実験により AUC の推移が解釈可能な範囲に収まることを確認する．収まらない場合は学習率を 0.01 へ上げ，40 ラウンドで概ね収束する設定へ切り替える（それでも CPU で一晩程度で完了する）．

5 実験条件

5.1 基本設定

表 1　基本となる学習・環境設定

項目	値
データセット	MNIST（10 クラス分類）
モデル	小型 CNN（Fang Table 7a 相当，または縮小版）
クライアント数 n	20（全クライアントが毎ラウンド参加）
通信ラウンド数	30（パイロットの結果により調整）
ローカルエポック数	1
バッチサイズ	32
学習率	0.001（収束確認用セットのみ 0.01）
最適化	素朴な SGD
実装	Python / PyTorch / Flower（既存基盤を再利用）
実行環境	CPU

5.2 主スイープ（最小構成）

表 2　主スイープの条件。検知器は同一の実行ログから事後的にすべて評価できる

軸	水準
Non-IID 度 α	100, 0.5, 0.1
攻撃	符号反転（A1），ALIE（A2）
悪性割合 f/n	0.2（固定，$f = 4$）
検知器	D1，D2，D3（事後評価，追加コストなし）
trial	3（データ分割・初期値・悪性選出に連動）

主スイープの実行数は $3 \times 2 \times 3 = 18$ 回．加えて，spread の基準値を得るための攻撃なし条件（$3\,\alpha \times 3\,\text{trial} = 9$ 回）を実行し，合計 27 回．1 回あたり CPU で数分であり，一晩以内に完了する．

5.3 発展課題（スケジュールに余裕がある場合のみ）

- min-max 攻撃（A3）を攻撃軸に追加（+9 回）．
- 悪性割合を $\{0.1, 0.3\}$ に拡張（+36 回）．
- 検知後に集約（Median，Trimmed Mean，Krum）まで行い，検知フィルタの有無で最終防御効用がどう変わるかを比較．
- 代表 4–6 セルを学習率 0.01・200 ラウンドで収束させ，検知失敗と精度低下の対応を確認．

6 実装方針

6.1 既存資産と新規作業の対応

本計画は，これまでに Fang 再現実験で構築した実験基盤の大部分をそのまま再利用する．変わるのは「何を測り，何を図示するか」であり，FL の学習・攻撃機構そのものは変更しない．既存資産と新規作業の対応を表 3 に示す．新規に書くコードは合計でおよそ 100 行であり，既に完了している 760 回スイープの実装規模と比べて小さい差分である．

表 3　既存資産と新規作業の対応

要素	状態	新規作業量
Flower + PyTorch の FL ループ	完成	0
FedAvg / Median / Trimmed Mean / Krum / Bulyan	`aggregation.py` に実装済み	0（関連研究で用いるのみ）
full knowledge 攻撃のサーバ側模擬・悪性選出	`attack.py` / `malicious.py` に実装済み	0
trial 連動の再現性（分割・初期値・悪性選出）	実装済み	0
作図・集計基盤	`regen_ch7_assets.py` 等が完成	微修正
Non-IID 分割	Fang 式は実装済み	ディリクレ分割へ差し替え，約 20 行
集約直前フックでの更新ベクトル要約ログ	分析計画書で設計済み	約 20 行
攻撃 2 種（符号反転／ ALIE）	新規	各 1–5 行
検知器 3 種（D1 ／ D2 ／ D3）	新規	各 5–15 行
評価指標（ROC-AUC，AP，recall@f）	新規	`scikit-learn` 呼び出し，約 10 行
新規コードの合計		約 100 行

6.2 新規に導入する数式の実装規模

§4.2–§4.5 の式は記法の密度に対して実装は数行で済む．実装が煩雑な Krum の $n - f - 2$ 近傍距離和や Yin の座標中央値は既に実装済みであり，新規の式はいずれもそれより単純である．

表 4　新規に導入する量とその実装規模の目安

量	計算内容	実装規模
spread / push	平均ベクトルからのユークリッド距離の平均	各 1 行
push_ratio	上記 2 量の比	1 行
ALIE の $g^{\mathrm{mal}} = \mu - z\sigma$	正常更新の座標ごと平均・標準偏差から生成	2 行
D1（ノルム偏差）	ノルムと，その中央値との差の絶対値	2 行
D2（コサイン距離）	1−（座標中央値ベクトルとのコサイン類似度）	1 行
D3（2 クラスタリング）	特徴量への k-means（$k = 2$, `scikit-learn`）	3–5 行
D4（スペクトル残差）	第 1 右特異ベクトルへの射影の 2 乗（`numpy.linalg.svd`）	3 行
ROC-AUC 等	`scikit-learn` の関数呼び出し（導出不要）	1 行

新たに理解が必要なのは ROC ／ AUC ／ precision–recall という分類器評価の考え方と，「なぜ検知 AUC が本研究の妥当な指標か」を説明できることであり，機械学習の基礎の範囲にとどまる．

6.3 実装手順

1. **データ分割：** Fang らのグループ分割を，ディリクレ分割（`flwr-datasets` もしくは自作の約 20 行）へ差し替える．クライアント別のクラス数ヒストグラムで分割結果を目視確認する．
2. **ロギング：** 集約直前フック（攻撃適用後・集約前）で，各クライアントについて $\|g_i\|_2$，D2 スコア，悪性フラグ y_i，および spread，push，push_ratio をラウンドごとに CSV へ追記する．更新ベクトル全体は保存せず要約統計のみ．
3. **検知器：** D1–D3 を各 5–20 行程度で実装し，`test_detector.py` を追加する（既存の `test_*.py` と同じ方針）．
4. **攻撃：** 符号反転と ALIE を追加する（既存のスケーリング攻撃を流用可能）．
5. **評価：** ROC-AUC・AP・recall@f は `scikit-learn` で後処理として計算する．検知器のハイパーパラメータに依存しないよう，主指標は閾値非依存の量に統一する．
6. **作図：** 既存の `regen_ch7_assets.py` 系の作図基盤を再利用する．

7 スケジュールと判定ゲート

目標は **11 月末までに実験完了および本文ドラフト完成**．以下は 13 週間の計画で，11 月後半の約 2 週間をバッファとする．卒業論文の第 1–3 章（FL の基礎，攻撃の分類，防御の分類，関連研究）は既存の `sotsuron.tex` をおおむね流用できるため，新規に執筆するのは方法・結果・考察の各章である．

表 5　週次スケジュール（2026 年 9 月 1 日–11 月 30 日）

週	期間	主な作業	ゲート／中間成果
1	9/1–9/7	ディリクレ分割へ差し替え，クライアント別クラス分布の確認．集約直前フックに検知用ロギングを追加	
2	9/8–9/14	検知器 D1–D3 の実装と `test_detector.py`．符号反転・ALIE 攻撃の実装	
3	9/15–9/21	パイロット実行（$n = 20$，30 ラウンド，ALIE，$\alpha = 0.1$）．AUC のラウンド推移を確認	**G1：** 未収束でも検知評価が成立するか．不成立なら学習率 0.01・40 ラウンドへ切替
4	9/22–9/28	実験グリッドの最終確定と**凍結**．集計・作図スクリプトの整備	**G2：** グリッド凍結．以降は条件を追加しない
5	9/29–10/5	主スイープ 27 回の実行，生データ確認と再実行	主スイープ完了
6	10/6–10/12	図表の生成．方法章の執筆	方法章ドラフト
7	10/13–10/19	結果章の執筆．考察の骨子（push_ratio と AUC の関係）を固める	**G3：** 主結果が 1 枚の図で言えるか．言えれば発展課題に着手
8	10/20–10/26	考察章の執筆．第 1–3 章を本テーマに合わせて改稿	
9	10/27–11/2	全体の通し読み．序論・結論の書き直し．（先行していれば min-max 攻撃を追加）	全章ドラフト
10	11/3–11/9	指導教員レビューと修正	レビュー反映
11	11/10–11/16	図の清書，参考文献の整備，体裁	
12–13	11/17–11/30	バッファ（追加実験・加筆・予稿作成）	完成

最小構成とフル構成の線引き：第 7 週のゲート G3 の時点で計画どおりに進んでいれば発展課題（min-max 攻撃，悪性割合の拡張，検知＋集約の比較）へ進む．遅れている場合は最小構成（主スイープ 27 回，検知器 3 種，攻撃 2 種，α 3 水準）で確定させ，発展課題はすべて「今後の課題」に回す．

8 想定される成果と妥当性への脅威

8.1 想定される成果

- 3 種の検知器について，Non-IID 度と攻撃ステルス性の 2 軸で整理した検知性能マップ．
- 「push_ratio $\lesssim 1$ で検知が破綻する」という定量的な破綻境界，および Non-IID が push_ratio を押し下げることによる検知性能低下の実証．
- 粗い攻撃（符号反転）とステルス攻撃（ALIE）の間で，どの検知器の優位性が逆転するかの整理．
- 主結果がきれいな境界を示さない場合でも，「明確な閾値はなく緩やかに劣化する」という形で問いには答えられる．

8.2 妥当性への脅威（考察で明示する）

- **規模：** 20 クライアント・MNIST の小規模設定であり，cross-device FL（多数・低参加率）への一般化は限定的．
- **未収束モデル：** 検知性能をラウンドの関数として報告し，収束サブセットでの確認を併用することで補う．
- **正解ラベルの分離：** 正解は評価専用であり検知器へ漏らさないことを実装で担保する．
- **検知器のハイパーパラメータ：** 主指標を ROC 等の閾値非依存量に統一し，$k = 2$，スペクトルは第 1 成分に固定する．
- **「検知」と「防御」の差：** 防御効用指標 $\|\hat{g} - \bar{g}_B\|$ を併記して橋渡しする．
- **脅威モデルの楽観性：** full knowledge・悪性集合固定という前提を明示し，Shejwalkar ら [15] の批判を踏まえて「上限性能の評価」と位置づける．

9 卒業論文の構成案

1. **序論：** FL とそのセキュリティ課題，本研究の問いと貢献．
2. **事前知識：** FL と FedAvg，Poisoning Attack の分類，防御の 3 分類（異常検知・ロバスト集約・摂動機構），本研究が扱う異常検知の位置づけ．（既存 `sotsuron.tex` を流用・圧縮）
3. **関連研究：** ロバスト集約（Krum，Median，Trimmed Mean，Bulyan），集約手法を突く攻撃（Fang，ALIE，min-max，IPM），検知型防御（FoolsGold，FLAME，FLDetector，学習型検知），Non-IID の扱い．本研究との差分を明示．
4. **提案する評価枠組み：** 脅威モデル，検知器 D1–D3，攻撃 A1–A2，ディリクレ Non-IID，評価指標（ROC 等，spread/push_ratio）．
5. **実験：** 設定，主スイープ，パイロットとゲート G1．
6. **結果：** 検知性能マップ，AUC 対ラウンド，push_ratio–AUC 主図，防御効用．
7. **考察：** 中心仮説の検証，検知器間の優劣の逆転，Non-IID の影響の機構，妥当性への脅威．
8. **結論と今後の課題：** 得られた知見，min-max 攻撃・大規模化・検知＋集約の統合など．

参考文献

[1] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, Neil Zhenqiang Gong, “Local Model Poisoning Attacks to Byzantine-Robust Federated Learning,” Proceedings of the 29th USENIX Security Symposium (USENIX Security 2020), pp. 1605–1622, 2020.

[2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, Blaise Aguera y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 1273–1282, 2017.

[3] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, Julien Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] Dong Yin, Yudong Chen, Kannan Ramchandran, Peter Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates," Proceedings of the 35th International Conference on Machine Learning (ICML), 2018.

[5] El Mahdi El Mhamdi, Rachid Guerraoui, Sébastien Rouault, "The Hidden Vulnerability of Distributed Learning in Byzantium," Proceedings of the 35th International Conference on Machine Learning (ICML), 2018.

[6] Gilad Baruch, Moran Baruch, Yoav Goldberg, "A Little Is Enough: Circumventing Defenses For Distributed Learning," Advances in Neural Information Processing Systems (NeurIPS), 2019.

[7] Virat Shejwalkar, Amir Houmansadr, "Manipulating the Byzantine: Optimizing Model Poisoning Attacks and Defenses for Federated Learning," Network and Distributed System Security Symposium (NDSS), 2021.

[8] Cong Xie, Oluwasanmi Koyejo, Indranil Gupta, "Fall of Empires: Breaking Byzantine-tolerant SGD by Inner Product Manipulation," Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI), 2019.

[9] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, Seraphin Calo, "Analyzing Federated Learning through an Adversarial Lens," Proceedings of the 36th International Conference on Machine Learning (ICML), 2019.

[10] Clement Fung, Chris J. M. Yoon, Ivan Beschastnikh, "The Limitations of Federated Learning in Sybil Settings," 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID), 2020.

[11] Suyi Li, Yong Cheng, Wei Wang, Yang Liu, Tianjian Chen, "Learning to Detect Malicious Clients for Robust Federated Learning," arXiv preprint arXiv:2002.00211, 2020.

[12] Thien Duc Nguyen et al., "FLAME: Taming Backdoors in Federated Learning," 31st USENIX Security Symposium (USENIX Security 2022), pp. 1415–1432, 2022.

[13] Zaixi Zhang, Xiaoyu Cao, Jinyuan Jia, Neil Zhenqiang Gong, "FLDetector: Defending Federated Learning Against Model Poisoning Attacks via Detecting Malicious Clients," Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2022.

[14] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, "FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping," Network and Distributed System Security Symposium (NDSS), 2021.

[15] Virat Shejwalkar, Amir Houmansadr, Peter Kairouz, Daniel Ramage, "Back to the Drawing Board: A Critical Evaluation of Poisoning Attacks on Production Federated Learning," IEEE Symposium on Security and Privacy (S&P), 2022.

[16] Tzu-Ming Harry Hsu, Hang Qi, Matthew Brown, "Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification," arXiv preprint arXiv:1909.06335, 2019.

[17] Qinbin Li, Yiqun Diao, Quan Chen, Bingsheng He, "Federated Learning on Non-IID Data Silos: An Experimental Study," Proceedings of the 38th IEEE International Conference on Data Engineering (ICDE), 2022.

[18] Shiqi Shen, Shruti Tople, Prateek Saxena, "Auror: Defending Against Poisoning Attacks in Collaborative Deep Learning Systems," Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC), 2016.

[19] Felix Sattler, Klaus-Robert Müller, Thomas Wiegand, Wojciech Samek, "On the Byzantine Robustness of Clustered Federated Learning," IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2020.

[20] Brandon Tran, Jerry Li, Aleksander Madry, "Spectral Signatures in Backdoor Attacks," Advances in Neural Information Processing Systems (NeurIPS), 2018.

[21] Lingjuan Lyu, Han Yu, Xingjun Ma, Chen Chen, Lichao Sun, Jun Zhao, Qiang Yang, Philip S. Yu, "Privacy and Robustness in Federated Learning: Attacks and Defenses," IEEE Transactions on Neural Networks and Learning Systems (TNNLS), 2022.

[22] Subhash Sagar, Chang-Tsun Li, Seng W. Loke, Jinho Choi, "Poisoning Attacks and Defenses in Federated Learning: A Survey," IEEE Security & Privacy, 2023.